

# Package ‘rfsnEfficiency’

July 3, 2026

**Title** Risk-Reduction Efficacy and Efficiency of Risk-Management Configurations

**Version** 0.0.0.9000

**Description** A general, on-demand toolkit for evaluating risk-management or safety-net configurations. From user-supplied end-of-season outcome draws (ideally one element per agent) and program parameters (cost, premium, subsidy, and an indemnification rule that may be a function), it builds a configurable outcome (revenue, profit, margin, or a custom / cost-side measure) relative to a no-program baseline and computes efficacy scores (mean lift, variability reduction) and a risk-reduction efficiency ratio, with comparisons across regimes or scenarios. Implements the risk-reduction-efficiency metric of Tsiboe et al. (2025).

**License** GPL (>= 3)

**Encoding** UTF-8

**LazyData** true

**Imports** data.table,  
stats,  
utils

**Suggests** testthat (>= 3.0.0),  
piggyback,  
usethis,  
knitr,  
rmarkdown

**VignetteBuilder** knitr

**Config/testthat/edition** 3

**Roxygen** list(markdown = TRUE)

**RoxygenNote** 7.3.3

**Depends** R (>= 3.5)

## Contents

|                                       |   |
|---------------------------------------|---|
| assemble_scenario_outcomes . . . . .  | 2 |
| balance_scenarios . . . . .           | 3 |
| build_agent_outcomes . . . . .        | 4 |
| build_outcome . . . . .               | 5 |
| calc_insurance_payment_rate . . . . . | 6 |
| compute_efficiency_scores . . . . .   | 7 |

|                                             |    |
|---------------------------------------------|----|
| efficiency_score_columns . . . . .          | 8  |
| efficiency_score_crosswalk . . . . .        | 9  |
| explode_draws . . . . .                     | 9  |
| income_transfer_score . . . . .             | 10 |
| regime_deltas . . . . .                     | 10 |
| risk_reduction_efficiency . . . . .         | 11 |
| run_efficiency_analysis . . . . .           | 12 |
| score_dictionary . . . . .                  | 13 |
| score_scenarios . . . . .                   | 13 |
| simulate_basic_policy_net_benefit . . . . . | 14 |
| simulate_example_outcomes . . . . .         | 15 |
| simulate_supplemental_net_benefit . . . . . | 16 |
| summarise_scores . . . . .                  | 17 |
| toy_outcomes . . . . .                      | 18 |
| trim_mean . . . . .                         | 19 |
| variability_reduction_score . . . . .       | 19 |

|              |           |
|--------------|-----------|
| <b>Index</b> | <b>21</b> |
|--------------|-----------|

---

assemble\_scenario\_outcomes

*Assemble scenario outcome columns from a baseline plus a component recipe*

---

## Description

Builds one outcome column per scenario as  $\text{baseline} + \text{sum}(\text{scale} * \text{component})$ , driven by a long recipe table. Each recipe row adds one component column (optionally scaled by another column, e.g. an adoption rate) onto the named scenario; scenarios with no component rows (or a single NA component) equal the baseline. Analysis-agnostic: the scenario design lives entirely in recipe.

## Usage

```
assemble_scenario_outcomes(data, baseline, recipe)
```

## Arguments

|          |                                                                                                                                                                                |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data     | data.table containing baseline and every component/scale column referenced by recipe.                                                                                          |
| baseline | name of the baseline outcome column added to every scenario.                                                                                                                   |
| recipe   | data.table with columns scenario (output column name), component (column to add; NA = baseline only), and scale (optional column to multiply the component by; NA = unscaled). |

## Value

data (modified in place) with one new column per unique scenario.

---

|                   |                                                                                 |
|-------------------|---------------------------------------------------------------------------------|
| balance_scenarios | <i>Balance a scored panel across scenario cells by pruning incomplete units</i> |
|-------------------|---------------------------------------------------------------------------------|

---

## Description

Guards cross-scenario comparisons against a hidden composition confound: a unit (e.g. a producer) should contribute to *every* scenario cell (e.g. every policy environment x program combination x participation), otherwise differences across cells can mix treatment with sample composition. This checks whether each unit is present – and, if `require_finite` is given, has finite values for those metric columns – in every required cell, and (by default) prunes to the units that are, returning a rebalanced panel.

## Usage

```
balance_scenarios(
  dt,
  unit,
  cell,
  require_finite = NULL,
  allow_missing = NULL,
  prun_to_rebalance = TRUE,
  min_retention = 0.5,
  weight = NULL,
  verbose = TRUE
)
```

## Arguments

|                                |                                                                                                                                                                                                                         |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dt</code>                | data.table with one row per unit x cell, plus the metric columns.                                                                                                                                                       |
| <code>unit</code>              | character vector of columns identifying a unit (the thing that must be common across cells; carried whole when kept).                                                                                                   |
| <code>cell</code>              | character vector of columns defining a scenario cell.                                                                                                                                                                   |
| <code>require_finite</code>    | optional character vector of metric columns that must be finite for a row to count as covering its cell. NULL (default) requires only row presence.                                                                     |
| <code>allow_missing</code>     | optional named list mapping cell columns to value(s); cells whose columns all match are dropped from the required grid (a unit is not penalised for missing them).                                                      |
| <code>prun_to_rebalance</code> | if TRUE (default) drop units that do not cover the full required grid and return the rebalanced panel; if FALSE return dt unchanged (report-only / dry run).                                                            |
| <code>min_retention</code>     | when pruning, <code>stop()</code> if the retained share (weighted by <code>weight</code> if supplied, else by unit count) falls below this fraction – heavy pruning signals a structural problem, not routine trimming. |
| <code>weight</code>            | optional per-unit weight column used for the retention share and the report.                                                                                                                                            |
| <code>verbose</code>           | if TRUE (default) message a one-line retention summary.                                                                                                                                                                 |

## Details

The required grid is the set of observed cell combinations minus any listed in `allow_missing` (for structurally absent combinations, e.g. a base arm that has no adoption pathway). Pruning at the finest unit level means every coarser aggregation (subgroup) inherits the balance automatically.

## Value

a `data.table`: the rebalanced panel (or `dt` unchanged when `prun_to_rebalance = FALSE`), with a `"balance_report"` attribute (per-cell valid-unit counts before, uniform count after, and overall retention).

---

|                                   |                                                                                        |
|-----------------------------------|----------------------------------------------------------------------------------------|
| <code>build_agent_outcomes</code> | <i>Build per-agent baseline/treated outcomes from end-of-season draws + parameters</i> |
|-----------------------------------|----------------------------------------------------------------------------------------|

---

## Description

The flexible entry point: hand in each agent's underlying end-of-season outcomes and the program parameters, and get back stacked baseline/treated outcome draws ready for `compute_efficiency_scores()`.

## Usage

```
build_agent_outcomes(
  agents,
  indemnity,
  outcome = "outcome",
  cost = 0,
  premium = 0,
  subsidy = 0,
  agent_id_col = "agent_id",
  extra_cols = NULL,
  floor_at = 0
)
```

## Arguments

|                                     |                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>agents</code>                 | a list with one element per agent. Each element is either a numeric vector of the base end-of-season outcome (draws), or a <code>data.frame/data.table</code> of draws (rows = crop years / Monte Carlo draws) containing at least the outcome column. List names, if present, become agent ids.                                                                                            |
| <code>indemnity</code>              | indemnification rule: a <code>function(agent_df)</code> returning a per-draw indemnity vector, OR a column name in each agent's data, OR a numeric scalar/vector. Indemnity is ADDED to the treated outcome.                                                                                                                                                                                |
| <code>outcome</code>                | name of the base end-of-season outcome column (default "outcome"); ignored when an agent element is a plain numeric vector.                                                                                                                                                                                                                                                                 |
| <code>cost, premium, subsidy</code> | program parameters, each a <code>function(agent_df)</code> , a column name, or a numeric scalar/vector. <code>cost</code> is subtracted from the base outcome (enters both baseline and treated). <code>premium</code> is the producer-paid premium, subtracted from the treated outcome. <code>subsidy</code> is carried through for program-context summaries (not added to the outcome). |

|              |                                                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| agent_id_col | name of the agent id column in the output (default "agent_id").                                                                                                       |
| extra_cols   | optional character vector of additional per-draw columns to carry through from each agent's data (e.g., "commodity_code", "combination", "regime", "commodity_year"). |
| floor_at     | numeric floor applied to both outcome series (default 0; NA = none).                                                                                                  |

## Details

$$outcome\_baseline = outcome - cost$$

$$outcome\_treated = outcome\_baseline + indemnity - producer\_premium$$

## Value

a data.table stacked over agents with agent\_id, draw, outcome\_baseline, outcome\_treated, and the resolved cost, producer\_premium, subsidy, indemnity columns (plus extra\_cols).

## Examples

```
# Two agents, each with 100 end-of-season revenue draws; indemnity is a
# function paying 70% of the shortfall below a 600 guarantee.
set.seed(1)
agents <- list(
  cornA = data.frame(outcome = pmax(0, rnorm(100, 600, 150))),
  cornB = data.frame(outcome = pmax(0, rnorm(100, 550, 200)))
)
indem <- function(df) pmax(0, 600 - df$outcome) * 0.70
d <- build_agent_outcomes(agents, indemnity = indem, premium = 25, subsidy = 0.62)
compute_efficiency_scores(d, by = "agent_id")[, .(agent_id, mean_index, risk_index, efficiency)]
```

---

|               |                                                                      |
|---------------|----------------------------------------------------------------------|
| build_outcome | <i>Construct the analysis outcome and its no-safety-net baseline</i> |
|---------------|----------------------------------------------------------------------|

---

## Description

Construct the analysis outcome and its no-safety-net baseline

## Usage

```
build_outcome(
  data,
  base_value = "revenue",
  cost = NULL,
  transfers = "indemnity",
  cost_protection = NULL,
  premium = "producer_premium",
  floor_at = 0
)
```

**Arguments**

|                 |                                                                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data            | data.frame/data.table with the component columns.                                                                                                                                                        |
| base_value      | column name of the gross pre-safety-net outcome (e.g., "revenue"; for profit pass revenue plus a cost).                                                                                                  |
| cost            | optional column subtracted from base_value. May be a stochastic per-observation series so cost risk enters the outcome (enabling cost-side products). NULL = revenue; a production-cost column = profit. |
| transfers       | character vector of revenue/yield-side payout columns ADDED to the treated outcome (e.g., c("indemnity", "title1_payment")). NULL for products that provide no revenue-side protection.                  |
| cost_protection | character vector of cost-side payout columns (a product that pays when costs rise) ADDED to the treated outcome. NULL if none.                                                                           |
| premium         | column SUBTRACTED from the treated outcome (producer-paid premium for whichever product(s) are active); NULL to ignore.                                                                                  |
| floor_at        | numeric floor applied to both series (default 0; NA = none).                                                                                                                                             |

**Value**

data with new columns outcome\_baseline and outcome\_treated.

**Examples**

```
## Not run:
build_outcome(d, base_value = "revenue", cost = NULL)           # revenue
build_outcome(d, base_value = "revenue", cost = "input_cost")   # profit
build_outcome(d, "revenue", cost = "input_cost", transfers = NULL, # cost-only
              cost_protection = "margin_indemnity", premium = "margin_premium")

## End(Not run)
```

---

calc\_insurance\_payment\_rate

*Calculate an insurance payment rate (indemnification)*

---

**Description**

Computes a payment rate in  $[0, 1]$  from expected and realized yields plus price inputs, and returns the intermediate guarantee / revenue-to-count quantities. Vectorized over its inputs.

**Usage**

```
calc_insurance_payment_rate(
  expected_yield,
  final_yield,
  harvest_price,
  price_election_amount,
  projected_price,
  trigger_index,
  coverage_range = 1,
  insurance_plan_code
)
```

**Arguments**

expected\_yield Numeric. Expected yield level (e.g. APH / approved yield).  
 final\_yield Numeric. Realized harvested yield.  
 harvest\_price Numeric. Market price at harvest (used for revenue-to-count in some plans).  
 price\_election\_amount  
                     Numeric. Elected price at sign-up (typically projected\_price \* price\_election\_percent).  
 projected\_price  
                     Numeric. Projected / initial price used for guarantee construction and caps.  
 trigger\_index Numeric. Coverage trigger as a proportion (e.g. the coverage level in  $[0, 1]$ ).  
 coverage\_range Numeric. Coverage range width; default 1.  
 insurance\_plan\_code  
                     Character or numeric. Plan code that determines whether the guarantee may use  
                     harvest price and whether revenue-to-count uses the harvest or the elected price.

**Details**

Plan-code logic is implementation-specific: insurance\_plan\_code membership tests decide (a) whether harvest price can raise the guarantee, and (b) whether revenue-to-count uses harvest\_price or price\_election\_amount. Ensure your plan-code conventions match those encoded here.

**Value**

A named list with new\_price\_election\_amount, new\_insurance\_guarantee, revenue\_to\_count, and payment\_rate.

**See Also**

[simulate\\_basic\\_policy\\_net\\_benefit\(\)](#), [simulate\\_supplemental\\_net\\_benefit\(\)](#)

---

compute\_efficiency\_scores

*Compute efficacy and efficiency scores from a generic outcome*

---

**Description**

Compute efficacy and efficiency scores from a generic outcome

**Usage**

```
compute_efficiency_scores(
  data,
  by,
  outcome_baseline = "outcome_baseline",
  outcome_treated = "outcome_treated",
  its_clamp = c(0.1, 2),
  vrs_clamp = c(0.1, 2),
  eff_deadband = 0.05,
  eff_cap = NULL
)
```

**Arguments**

|                                   |                                                                                                                                                                                                                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data                              | data.frame/data.table with grouping columns and the two outcome series (outcome_baseline, outcome_treated), one row per draw/year within each group.                                                                                                                             |
| by                                | character vector of grouping columns (the unit the scores describe, e.g., agent identifiers x combination).                                                                                                                                                                      |
| outcome_baseline, outcome_treated | column names of the no-safety-net and with-safety-net outcome series.                                                                                                                                                                                                            |
| its_clamp                         | length-2 numeric c(lower, upper) winsorizing the income transfer score (mean_index); passed to income_transfer_score(). NULL leaves it unclamped. Default c(0.10, 2).                                                                                                            |
| vrs_clamp                         | length-2 numeric c(lower, upper) winsorizing every variability reduction score (cv_index and the downside/shortfall flavors); passed to variability_reduction_score(). NULL leaves them unclamped. Default c(0.10, 2). Independent of its_clamp.                                 |
| eff_deadband                      | minimum proportional mean gain (mean_index - 1) required before an efficiency (RRER) score is nonzero; passed to risk_reduction_efficiency(). Guards the ITS $\rightarrow 1$ singularity and bounds efficiency* at $1 / \text{eff\_deadband}$ . Default 0.05; 0 for raw Eq. (3). |
| eff_cap                           | optional hard upper bound on the efficiency scores (NULL = none), passed to risk_reduction_efficiency().                                                                                                                                                                         |

**Value**

data.table, one row per by group, with moments (\*\_base/\*\_sn), relative indices (\*\_index), headline scores (mean\_index, risk\_index, efficiency), flags, percent transforms, and the downside/shortfall flavors.

---

efficiency\_score\_columns

*Canonical column names emitted by compute\_efficiency\_scores()*

---

**Description**

Returns the score / indicator column names produced by [compute\\_efficiency\\_scores](#), grouped by how they should be summarised: continuous scores (indices, efficiency, percent transforms), the baseline/safety-net moments, and the logical flags. Used as the default column set by [summarise\\_scores](#) and available for callers that want to summarise or difference the same set.

**Usage**

```
efficiency_score_columns()
```

**Value**

named list with character vectors scores, moments, and flags.



---

efficiency\_score\_crosswalk

*Crosswalk: intuitive names <-> legacy / source-pipeline column names*

---

### Description

Maps the package's intuitive score names to the terse column names used by earlier implementations of the same metrics (e.g., its, Relcv, sner1).

### Usage

```
efficiency_score_crosswalk()
```

### Value

```
data.frame(score, legacy, meaning).
```

---

explode\_draws

*Explode nested draw list-columns to long form*

---

### Description

Unnests one or more equal-length list-columns (e.g. per-agent Monte-Carlo draw vectors) into a long table with one row per input row x draw element. Every column that is neither a draw column nor dropped is carried (recycled) onto the expanded rows. Analysis-agnostic.

### Usage

```
explode_draws(
  data,
  draw_columns,
  value_names = draw_columns,
  drop = character(0)
)
```

### Arguments

|              |                                                                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| data         | data.frame/data.table with the list-columns named in draw_columns; all list-columns must have the same length within a row. |
| draw_columns | character vector of list-column names to unnest.                                                                            |
| value_names  | names for the unnested output columns (defaults to draw_columns).                                                           |
| drop         | character vector of columns to discard (e.g. draw metadata not needed downstream).                                          |

### Value

a data.table, long over draws.

---

|                       |                                    |
|-----------------------|------------------------------------|
| income_transfer_score | <i>Income Transfer Score (ITS)</i> |
|-----------------------|------------------------------------|

---

**Description**

Ratio of the expected outcome with the safety net to the expected outcome without it. ITS > 1 indicates the configuration raises the mean outcome. (Eq. 2 in Tsiboe et al. 2025.)

**Usage**

```
income_transfer_score(treated_mean, baseline_mean, clamp = c(0.1, 2))
```

**Arguments**

|                             |                                                                                                                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| treated_mean, baseline_mean | numeric vectors of mean outcomes with and without the safety net.                                                                                                                         |
| clamp                       | optional length-2 numeric c(lower, upper) to winsorize the ratio into (guards against a near-zero baseline mean blowing the ratio up). NULL leaves the raw ratio. Defaults to c(0.10, 2). |

**Value**

numeric vector (non-finite ratios returned as NA; finite values clamped to clamp when supplied).

---

|               |                                                          |
|---------------|----------------------------------------------------------|
| regime_deltas | <i>Regime deltas (alt - ref) for every numeric score</i> |
|---------------|----------------------------------------------------------|

---

**Description**

Regime deltas (alt - ref) for every numeric score

**Usage**

```
regime_deltas(  
  scores,  
  by,  
  metrics = NULL,  
  regime_col = "regime",  
  ref = NULL,  
  alt = NULL  
)
```

**Arguments**

|            |                                                                                                            |
|------------|------------------------------------------------------------------------------------------------------------|
| scores     | data.table from compute_efficiency_scores() stacked over regimes (must include regime_col).                |
| by         | grouping columns identifying a comparable unit across regimes.                                             |
| metrics    | numeric score columns to difference (default: all numeric).                                                |
| regime_col | regime column name (default "regime").                                                                     |
| ref, alt   | reference and alternative regime labels. If NULL (default), the first two (sorted) regime values are used. |

**Value**

data.table with by, metric, the two regime columns, and delta.

---

risk\_reduction\_efficiency

*Risk Reduction Efficiency Ratio (RRER) – Equation (3)*

---

**Description**

$$RRER = \mathbb{I}(VRS < 1) \mathbb{I}(ITS > 1) \frac{1 - VRS}{ITS - 1}$$

**Usage**

```
risk_reduction_efficiency(its, vrs, deadband = 0.05, cap = NULL)
```

**Arguments**

|          |                                                                                                                                                                                                                                                         |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| its      | Income Transfer Score (from income_transfer_score()).                                                                                                                                                                                                   |
| vrs      | Variability Reduction Score (from variability_reduction_score()).                                                                                                                                                                                       |
| deadband | minimum proportional mean gain $ITS - 1$ required for a nonzero score; below it the efficiency is 0. Guards the $ITS \rightarrow 1$ singularity and bounds the score at $1 / \text{deadband}$ . Default 0.05; use 0 for the raw Equation (3) behaviour. |
| cap      | optional hard upper bound applied after the ratio (NULL = none).                                                                                                                                                                                        |

**Details**

The proportional variability reduction deflated by the proportional income (outcome) gain required to achieve it. It is 0 unless the configuration both lifts the mean and cuts variability ( $VRS < 1$ ); otherwise it ranges over  $[0, \text{Inf})$ , with larger values = more efficient risk mitigation. Exactly Equation (3) of Tsiboe et al. (2025).

Because the denominator  $ITS - 1$  vanishes as  $ITS \rightarrow 1$ , a vanishingly small mean gain can make the raw ratio explode. deadband guards this by requiring a minimum proportional gain ( $ITS - 1 \geq \text{deadband}$ ) before the ratio is taken; below it the score is 0. This also bounds the result: since  $VRS \geq 0$ , the maximum is  $1 / \text{deadband}$  (and  $(1 - \text{vrs\_floor}) / \text{deadband}$  when  $VRS$  is floored). An optional cap applies a further hard ceiling.

**Value**

numeric vector in  $[0, 1/\text{deadband}]$  (or  $[0, \text{cap}]$ ).

**Examples**

```
risk_reduction_efficiency(its = 1.05, vrs = 0.80) # both conditions met
risk_reduction_efficiency(its = 0.98, vrs = 0.80) # mean not lifted -> 0
risk_reduction_efficiency(its = 1.001, vrs = 0.50) # gain below deadband -> 0
```

---

run\_efficiency\_analysis

*Run an on-demand efficiency analysis*


---

## Description

High-level driver: given simulated outcomes (one row per draw/year x group, optionally tagged by regime/scenario), build the outcome if needed, compute the efficacy/efficiency scores per regime, and difference regimes.

## Usage

```
run_efficiency_analysis(
  data,
  by,
  regime_col = "regime",
  outcome_args = NULL,
  ref = NULL,
  alt = NULL
)
```

## Arguments

|              |                                                                                                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data         | data.frame/data.table of simulated outcomes. Must contain the by columns and either (outcome_baseline, outcome_treated) or the raw components that build_outcome() needs (then pass outcome_args). |
| by           | character vector of grouping columns (e.g., c("group_id", "scenario")).                                                                                                                            |
| regime_col   | regime column name (default "regime"); created as "all" if absent.                                                                                                                                 |
| outcome_args | named list of arguments forwarded to build_outcome() when the outcome columns are not already present (e.g., list(base_value = "outcome", transfers = "indemnity", premium = "premium")).          |
| ref, alt     | reference and alternative regime labels for the deltas. If NULL (default) and exactly/at least two regimes are present, the first two (sorted) are used.                                           |

## Value

list with scores (per regime) and deltas (alt - ref).

## Examples

```
# Self-contained: synthetic outcomes, no external data required.
d <- simulate_example_outcomes()
res <- run_efficiency_analysis(
  data = d, by = c("group_id", "scenario"),
  outcome_args = list(base_value = "outcome", transfers = "indemnity",
    premium = "premium"))
head(res$scores)
head(res$deltas)
```

---

|                  |                                       |
|------------------|---------------------------------------|
| score_dictionary | <i>Score and indicator dictionary</i> |
|------------------|---------------------------------------|

---

### Description

A committed look-up table explaining every column produced by `compute_efficiency_scores()` – the moments, relative indices, headline scores, efficiency ratios, flags, and percent transforms – plus the naming conventions used by `summarise_scores()` (`n_producers`, `med_*`, `pct_*`) and the report-pipeline `d_*` deltas. Use it to look up what a variable means, how it is calculated, and how to read it. Metrics follow Tsiboe et al. (2025).

### Usage

```
score_dictionary
```

### Format

A `data.frame` with one row per variable (or naming convention) and columns:

**variable** the column name (or `<...>` naming pattern) it documents.

**group** family: moment, index, score, flag, percent, summary, or delta.

**description** what the quantity is.

**calculation** how it is computed (in terms of the moments/indices).

**interpretation** how to read its value.

### Source

Built internally

---

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| score_scenarios | <i>Score many outcome columns against a common baseline</i> |
|-----------------|-------------------------------------------------------------|

---

### Description

Loops over `outcome_cols`, scoring each against `baseline_col` with `compute_efficiency_scores()` (draws = the rows within each by group), and row-binds the per-scenario scores. Optionally merges a `scenario_map` of labels and attaches a per-group weight. Analysis-agnostic wrapper.

### Usage

```
score_scenarios(
  data,
  outcome_cols,
  by,
  baseline_col = "revenue00",
  scenario_map = NULL,
  weight = NULL,
  scenario_col = "scenario_col"
)
```

**Arguments**

|              |                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------|
| data         | data.table with by, baseline_col, and every outcome_cols column; one row per group x draw.    |
| outcome_cols | character vector of treated outcome columns to score.                                         |
| by           | character vector of grouping columns (the scored unit; draws are the rows within each group). |
| baseline_col | name of the no-treatment baseline column.                                                     |
| scenario_map | optional data.table keyed by scenario_col to merge scenario labels onto the scores.           |
| weight       | optional name of a per-group weight column in data to carry onto the scores.                  |
| scenario_col | name of the column identifying the scored outcome column (default "scenario_col").            |

**Value**

a data.table of scores, one row per by group x scenario.

---

```
simulate_basic_policy_net_benefit
```

*Simulate a basic (individual) policy's per-draw net benefit*

---

**Description**

Adds an individual, loss-contingent indemnity and a scenario net benefit to a draw-level agent table. The indemnity is the M-13 payment rate applied to the simulated farm yield draw (yield\_farm) against the guarantee, times the underlying liability; it is triggered at the coverage level (full payout at total loss). Because the indemnity does not depend on the subsidy scenario it is computed once (guarded on underlying\_indemnity\_amount) and reused.

**Usage**

```
simulate_basic_policy_net_benefit(
  data,
  subsidy_amount,
  net_benefit_label,
  indemnity_label = NULL
)
```

**Arguments**

|                   |                                                                                                                                                                                                                                                                                 |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data              | data.table exploded to one row per agent x draw. Must contain underlying_liability_amount, underlying_total_premium_amount, approved_yield, yield_farm, harvest_price, projected_price, coverage_level_percent, insurance_plan_code, and the subsidy_amount column named below. |
| subsidy_amount    | name of the per-draw subsidy-amount column to add in.                                                                                                                                                                                                                           |
| net_benefit_label | name of the net-benefit column to create (indemnity - premium + subsidy).                                                                                                                                                                                                       |
| indemnity_label   | optional name for a copy of the per-draw indemnity to keep on data. NULL (default) leaves only underlying_indemnity_amount.                                                                                                                                                     |

**Value**

data (modified in place); invisibly, with underlying\_indemnity\_amount, net\_benefit\_label, and (if requested) indemnity\_label added.

---

```
simulate_example_outcomes
```

*Simulate a tiny example outcomes dataset (no external data required)*

---

**Description**

Generates synthetic per-draw outcomes for two regimes and a few scenarios so the package can be demonstrated, tested, and learned standalone. This is an ILLUSTRATIVE generator, not a calibrated model – the numbers carry no meaning.

**Usage**

```
simulate_example_outcomes(
  n_groups = 6,
  n_draws = 200,
  regimes = c("baseline", "reform"),
  seed = 1
)
```

**Arguments**

|          |                                                                |
|----------|----------------------------------------------------------------|
| n_groups | number of groups (e.g., units / commodities / portfolios).     |
| n_draws  | draws (years / Monte Carlo) per group per scenario per regime. |
| regimes  | character vector of regime labels.                             |
| seed     | RNG seed for reproducibility.                                  |

**Value**

a data.table with group\_id, scenario, regime, outcome, indemnity, and premium.

**Examples**

```
d <- simulate_example_outcomes(n_groups = 3, n_draws = 50)
head(d)
```

---

```
simulate_supplemental_net_benefit
```

*Simulate a supplemental (area) SCO/ECO endorsement's per-draw net benefit*

---

## Description

Adds an area-triggered, loss-contingent supplemental net benefit to a draw-level agent table. Supplemental liability is the underlying liability scaled to the endorsement band ( $\text{area\_loss\_start\_percent} - \text{area\_loss\_end\_percent}$ ); premium and subsidy are actuarial. The indemnity is the M-13 payment rate on the COUNTY (pool) draw ( $\text{final\_county\_yield}$  vs  $\text{expected\_county\_yield}$ ), so payouts follow the area outcome and basis risk is preserved. `trigger_index` is the band top (coverage ratio) and `coverage_range` the band width; the underlying `insurance_plan_code` selects yield vs revenue-to-count logic.

## Usage

```
simulate_supplemental_net_benefit(
  data,
  area_loss_start_percent,
  area_loss_end_percent,
  base_rate,
  rate_differential = NULL,
  subsidy_percent,
  net_benefit_label,
  indemnity_label = NULL
)
```

## Arguments

|                                                                           |                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data</code>                                                         | data.table exploded to one row per agent x draw. Must contain <code>underlying_liability_amount</code> , <code>coverage_level_percent</code> , <code>expected_county_yield</code> , <code>final_county_yield</code> , <code>harvest_price</code> , <code>projected_price</code> , <code>insurance_plan_code</code> , and the band/rate/subsidy columns named by the string arguments below. |
| <code>area_loss_start_percent</code> , <code>area_loss_end_percent</code> | column names of the band top and bottom (coverage ratios).                                                                                                                                                                                                                                                                                                                                  |
| <code>base_rate</code>                                                    | column name of the supplemental base premium rate.                                                                                                                                                                                                                                                                                                                                          |
| <code>rate_differential</code>                                            | optional column name of a multiplicative premium-rate differential (OBBBA recalibrations); NULL uses <code>base_rate</code> alone.                                                                                                                                                                                                                                                          |
| <code>subsidy_percent</code>                                              | column name of the supplemental subsidy share.                                                                                                                                                                                                                                                                                                                                              |
| <code>net_benefit_label</code>                                            | name of the net-benefit column to create ( $\text{indemnity} - \text{premium} + \text{subsidy}$ ).                                                                                                                                                                                                                                                                                          |
| <code>indemnity_label</code>                                              | optional name under which to keep a copy of the per-draw supplemental indemnity. NULL (default) drops it with the other intermediates.                                                                                                                                                                                                                                                      |



**Value**

data (modified in place); the intermediate supplemental columns are dropped, leaving `net_benefit_label` (and `indemnity_label` if requested).

---

|                               |                                                                    |
|-------------------------------|--------------------------------------------------------------------|
| <code>summarise_scores</code> | <i>Summarise producer-level scores into robust group summaries</i> |
|-------------------------------|--------------------------------------------------------------------|

---

**Description**

Collapses the per-producer score table from `compute_efficiency_scores()` to one row per by group. By default it summarises **every** score, indicator, percent transform, and moment the scorer produces: each continuous metric `m` yields a robust (trimmed) mean kept as `m` plus its median `med_m`, and each logical flag `f` yields the share (percent) of the group flagged, `pct_f`. A few producers with near-zero baseline revenue produce extreme indices, so the means are trimmed (see [trim\\_mean](#)) and the medians reported alongside as a robustness check. Every summary accepts an optional weight column, so results can be expressed per acre, per dollar of liability, or per simulation weight rather than per producer.

**Usage**

```
summarise_scores(
  dt,
  by,
  weight = NULL,
  lo = 0.005,
  hi = 0.995,
  metrics = NULL,
  flags = NULL,
  medians = TRUE,
  include_moments = TRUE
)
```

**Arguments**

|                              |                                                                                                                                                                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dt</code>              | data.table of per-producer scores (from <code>compute_efficiency_scores()</code> ).                                                                                                                                               |
| <code>by</code>              | character vector of grouping columns, or NULL for a single overall summary.                                                                                                                                                       |
| <code>weight</code>          | optional name of a numeric weight column in <code>dt</code> . NULL (default) weights every producer equally; when supplied, the trimmed means, medians, and flag shares are all weighted by it.                                   |
| <code>lo, hi</code>          | trimming quantile cut points forwarded to <a href="#">trim_mean</a> .                                                                                                                                                             |
| <code>metrics</code>         | optional character vector of continuous columns to summarise. NULL (default) uses the scores (and, if <code>include_moments</code> , moments) from <a href="#">efficiency_score_columns</a> that are present in <code>dt</code> . |
| <code>flags</code>           | optional character vector of logical/0-1 columns to report as <code>pct_*</code> shares. NULL (default) uses the flags from <a href="#">efficiency_score_columns</a> present in <code>dt</code> .                                 |
| <code>medians</code>         | whether to also report a <code>med_*</code> median for every metric (default TRUE).                                                                                                                                               |
| <code>include_moments</code> | whether to include the baseline/safety-net <code>*_base/*_sn</code> moments among the default metrics (default TRUE).                                                                                                             |

Details

Which columns are summarised is discovered from `efficiency_score_columns` intersected with the columns actually present, so non-score columns (identifiers, group codes, the weight) are ignored automatically. Pass `metrics/flags` to override.

Value

`data.table`, one row per by group, with `n_producers`, a trimmed mean for every metric, a `med_*` median for every metric (when medians), and a `pct_*` share for every flag.

See Also

`compute_efficiency_scores`, `efficiency_score_columns`, `trim_mean`

---

|              |                                                 |
|--------------|-------------------------------------------------|
| toy_outcomes | <i>Toy outcomes dataset for examples and CI</i> |
|--------------|-------------------------------------------------|

---

Description

A small, committed dataset so examples and the test suite run with no network access. Each insurance pool’s calibrated yields *across years* serve as its per-agent outcome draws; the indemnity and premium are illustrative (two regimes, baseline and reform) and carry no policy meaning.

Usage

toy\_outcomes

Format

A `data.table` with one row per pool x regime x year and columns:

- group\_id** insurance-pool identifier (the agent).
- regime** "baseline" or "reform".
- year** commodity year (the draw index).
- outcome** calibrated yield (the end-of-season outcome).
- indemnity** illustrative shortfall payment below the guarantee.
- premium** illustrative producer-paid premium.

Source

Built by `data-raw/scripts/build_toy_dataset.R` from the [USFarmSafetyNetLab calibrated\\_yield release](#).

---

|           |                                           |
|-----------|-------------------------------------------|
| trim_mean | <i>Trimmed (optionally weighted) mean</i> |
|-----------|-------------------------------------------|

---

**Description**

Mean of  $x$  after discarding the tails below the  $lo$  and above the  $hi$  quantiles. This guards headline score summaries against the handful of producers whose near-zero baseline revenue makes their indices explode and dominate a plain mean (the same trimming used in Tsiboe et al. 2025). Non-finite values are dropped before trimming. The tails are cut at the (unweighted)  $lo/hi$  sample quantiles of  $x$  – the extreme values are extreme regardless of weight – and, when `weights` is supplied, the retained values are then averaged with those weights. Equal weights therefore reproduce the unweighted trimmed mean exactly.

**Usage**

```
trim_mean(x, lo = 0.005, hi = 0.995, weights = NULL)
```

**Arguments**

- `x` numeric vector to summarise.
- `lo, hi` lower/upper quantile cut points (defaults 0.005 / 0.995).
- `weights` optional numeric vector of non-negative weights, the same length as `x`. `NULL` (default) gives the unweighted trimmed mean.

**Value**

length-one numeric; `NA_real_` when no finite values remain.

**Examples**

```
trim_mean(c(1, 2, 3, 4, 1000))
trim_mean(c(1, 2, 3, 4, 1000), weights = c(1, 1, 1, 1, 5))
```

---

|                             |                                          |
|-----------------------------|------------------------------------------|
| variability_reduction_score | <i>Variability Reduction Score (VRS)</i> |
|-----------------------------|------------------------------------------|

---

**Description**

Ratio of an outcome-variability measure with the safety net to the measure without it.  $VRS < 1$  indicates the configuration reduces variability. Works for ANY variability input (CV, downside semivariance, loss-conditional severity, ...), giving the headline and the robustness flavors.

**Usage**

```
variability_reduction_score(treated_var, baseline_var, clamp = c(0.1, 2))
```

**Arguments**

- `treated_var, baseline_var`  
numeric vectors of the variability measure with and without the safety net.
- `clamp`  
optional length-2 numeric `c(lower, upper)` to winsorize the ratio into (guards against a near-zero baseline variability blowing the ratio up). NULL leaves the raw ratio. Defaults to `c(0.10, 2)`.

**Value**

numeric vector (non-finite ratios returned as NA; finite values clamped to `clamp` when supplied).

# Index

## \* datasets

- score\_dictionary, [13](#)
- toy\_outcomes, [18](#)

assemble\_scenario\_outcomes, [2](#)

balance\_scenarios, [3](#)  
build\_agent\_outcomes, [4](#)  
build\_outcome, [5](#)

calc\_insurance\_payment\_rate, [6](#)  
compute\_efficiency\_scores, [7](#), [8](#), [18](#)

efficiency\_score\_columns, [8](#), [17](#), [18](#)  
efficiency\_score\_crosswalk, [9](#)  
explode\_draws, [9](#)

income\_transfer\_score, [10](#)

regime\_deltas, [10](#)  
risk\_reduction\_efficiency, [11](#)  
run\_efficiency\_analysis, [12](#)

score\_dictionary, [13](#)  
score\_scenarios, [13](#)  
simulate\_basic\_policy\_net\_benefit, [14](#)  
simulate\_basic\_policy\_net\_benefit(), [7](#)  
simulate\_example\_outcomes, [15](#)  
simulate\_supplemental\_net\_benefit, [16](#)  
simulate\_supplemental\_net\_benefit(), [7](#)  
summarise\_scores, [8](#), [17](#)

toy\_outcomes, [18](#)  
trim\_mean, [17](#), [18](#), [19](#)

variability\_reduction\_score, [19](#)